QCU ACADEMIC OS · PRE-CSS/JS CHECKPOINT · 2026

HTML5

Complete Tag & Concept Reference — everything you've practiced, plus everything you haven't yet

Builds on: WS101 Wk 2-3 | Leads to: CSS (Wk 4) + JS (Wk 5-6)

Format: Single-file, offline-readable | Live demos: Real, working examples

CHAPTER 00

Overview & the One Rule That Matters Most

This guide covers every HTML tag and concept from your BroCode practice file (`firstproj.html`), plus the tags and rules that weren't in that video but that you'll run into constantly once you start writing real pages — `fieldset`, `figure`, `details`, `section` vs `article`, global attributes, entities, and more. Wherever an example needs an actual image, audio, or video file, this guide uses real public URLs instead of a fake local path — so every demo below actually loads and plays when you open this file, instead of showing a broken-image icon.

Why This Comes Before CSS and JavaScript

HTML is the **structure** layer — what content exists and how it's organized. CSS (your next lesson) controls how that structure *looks*. JavaScript controls how it *behaves*. Both of those technologies constantly reference HTML elements by tag, id, or class — so gaps in your HTML foundation become confusing bugs later ("why won't my CSS selector work?" is almost always an HTML structure problem in disguise). Finish this reference, and CSS/JS will click faster.

The One Rule: Void vs. Paired Tags

Every single tag in HTML falls into exactly one of these two buckets. Get this straight now and half of your future "why isn't this working" moments disappear.

BUCKET 1

Void / Self-Closing Tags

Never have a closing tag. Never wrap content — they ARE the content. Writing `</br>` or `</hr>` is a beginner mistake.

<hr>
<img>
<input>
<meta>
<link>
<source>
<area>
<base>
<col>
<embed>
<track>
<wbr>

BUCKET 2

Paired / Container Tags

Always need a matching closing tag. They hold content or other tags between the open a close.

<html><head><body><title><p><h1>-<h6><a> <table><form><label><select><button><audio><video> .. most other tags

Reference Priority Map

Chapter	Topic	How Often You'll Use It	Difficulty
01	Document Anatomy	Every page	Easy
02	Text Content	Every page	Easy
03	Links & Navigation	Every page	Easy
04	Images / Audio / Video / iframe	Most pages	Medium
05	Lists & Tables	Most pages	Easy
06	Grouping & Semantic Layout	Every page	Medium
07	Forms	Most sites	Hardest
08	Global Attributes & Entities	Constantly, quietly	Easy
09	Accessibility & SEO	Professional habit	Medium

CHAPTER 01

Document Anatomy & Head Metadata

Every HTML page is one tree with a strict skeleton. Get this skeleton wrong and browsers guess how to fix it — usually wrong. This is the same boilerplate you already used in `firstproj.html`, explained piece by piece.

How a Browser Reads Your File, Top to Bottom

1. **`<!DOCTYPE html>`**
   Must be the very first line, no exceptions. Tells the browser "parse this as modern HTML5," not some legacy quirks-mode guess. It is NOT a real tag — it has no closing tag and takes no attributes.

2. **`<html lang="en">`**
   The root container for literally everything else. The `lang` attribute tells screen readers and translators what language the page is in.

3. **`<head>...</head>`**
   Invisible configuration zone. Nothing inside here is rendered on the page itself — it's metadata about the page, not content of the page.

4. **`<body>...</body>`**
   Everything the visitor actually sees and interacts with lives in here. Everything from Chapters 02-08 below goes inside `<body>`.

Head Metadata Reference

HTML

```
<head>
    <meta charset="UTF-8">
    <!-- Character encoding. Without this, special characters (· é © ñ)
can render as garbled boxes. VOID TAG. -->

    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <!-- Makes the page scale to the ACTUAL device width instead of
shrinking a desktop layout. VOID TAG. -->

    <meta name="description" content="A one-sentence summary shown in
Google search results.">
    <!-- Doesn't affect rendering at all — purely for search engines and
link previews. VOID TAG. -->

    <link rel="icon" href="images/favicon.png" type="image/png">
    <!-- Browser-tab icon. Resize the source image to a small square
(e.g. 32x32 or 96x96) first. VOID TAG. -->
```

```
    <link rel="stylesheet" href="style.css">
    <!-- This is how you'll attach your CSS file once you start that
lesson. Same <link> tag, different rel value. VOID TAG. -->

    <title>My Page Title</title>
    <!-- Text on the browser tab / bookmark name. PAIRED TAG. -->
</head>
```

Missing DOCTYPE	Old browsers fall back to "quirks mode," which changes box sizing and spacing rules unpredictably. Always include it, always as the first line.
Attribute typos	Duplicate attributes (writing `alt` twice on one <img>) are invalid — only the first is used, the rest are silently ignored. This was in your original file on both images.
Comment spacing	Comments must be written `<!-- text -->` with no space after the first `!`. `<! -- text -->` is invalid, even if some browsers tolerate it.

CHAPTER 02

Text Content — Headings, Paragraphs & Formatting

This is everything from your "text formatting" section, plus the tags that carry actual meaning instead of just a look.

Headings & Paragraphs — Live Demo

• RENDERED OUTPUT

This is an h1

This is an h3

This is a paragraph. One <h1> per page is the convention — it's your main title. Headings go h1 → h6 in order of importance, not by how big you want the text (that's CSS's job).

Presentational Tags vs. Semantic Tags

Your BroCode file used <b>, <i>, <u>, <big>, <small>, <tt>, <sub>, <sup>, <del>, and <mark>. Those all still work — but several have a semantic sibling that *looks* identical by default while carrying extra meaning that screen readers announce differently.

PRESENTATIONAL — just a look

<b> <i>

Bold/italic with zero implied meaning. Fine for stylistic-only text like a book title mentioned in passing.

SEMANTIC — meaning + look

** **

Renders bold/italic by default too, but tells assistive tech "this word is genuinely important / genuinely emphasized." Prefer these when the emphasis changes the sentence's meaning.

• RENDERED OUTPUT — ALL YOUR TEXT-FORMATTING TAGS

Bold · *Italic* · Underline · ~~Deleted~~ · Inserted · Sub · Sup

Big · Small · `Teletype` · Marked · **Strong** · *Emphasis*

> *"A blockquote — for quoting a chunk of another source."* — *cite names the source*

Inline quote: “a short quote” (browsers add the quotation marks automatically). Abbreviation: HTML (hover it).

Contact info goes in <address>.

HTML Entities — Typing Reserved Characters

Some characters are reserved by HTML itself (like < and >, since those define tags). To display them as literal text, you use an **entity** — a text code standing in for the symbol.

Entity	Renders as	Why you'd use it
`<`	<	Show a literal less-than sign without it being read as the start of a tag
`>`	>	Same idea, for greater-than
`&`	&	A literal ampersand (typing a raw & can break other entities)
`&quot;`	"	A literal double-quote inside an attribute string
` `	(a space)	A "non-breaking space" — keeps two words glued on the same line
`&copy;`	©	Copyright symbol — you already used this in your footer
`&hellip;`	…	An ellipsis, typed correctly instead of three periods

CHAPTER 03

Links & Navigation

The <a> (anchor) tag, every kind of destination it can point to, and the security detail most beginner tutorials skip.

Every Type of href, Live

● RENDERED OUTPUT — ALL LINKS BELOW ACTUALLY WORK

Absolute URL, opens in a new tab →

Same-page anchor link → jumps to Chapter 07 below

mailto: → opens the visitor's email app

tel: → opens the phone dialer on mobile

HTML

```
<!-- Relative path — points to a file in the SAME project folder, no
https:// needed -->
<a href="about.html">About Page</a>

<!-- Same-page anchor — jumps to any element whose id matches -->
<a href="#ch7">Jump to Forms</a>
<section id="ch7">...</section>

<!-- download attribute — forces the browser to download the file instead
of opening it.
     Only works reliably for files on your OWN site/origin, not random
external links. -->
<a href="resume.pdf" download>Download my resume</a>
```

The Security Detail Most Tutorials Skip

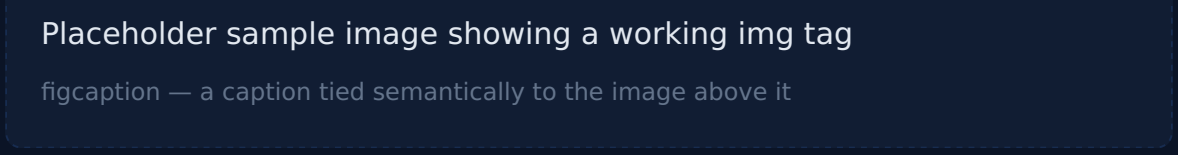
Any time you use `target="_blank"`, also add `rel="noopener noreferrer"`. Without it, the page you just linked to gets partial JavaScript access back to your original tab — a real, documented security/performance issue. It costs nothing to add and browsers don't do it for you automatically.

Images, Audio, Video & Embedded Pages

Same tags as your practice file — `img`, `audio`, `video` — plus `figure`/`figcaption` and `iframe`, which weren't in your BroCode video. Every demo below loads from a real public URL, so you'll actually see/hear/watch it play.

Images — Live Demo

• RENDERED OUTPUT



Placeholder sample image showing a working img tag

figcaption — a caption tied semantically to the image above it

HTML

```
<!-- Using your OWN local file (the syntax you'll use in your real
projects): -->
<img src="images/photo.jpg" alt="Describe what's in the photo"
width="280" height="180">

<!-- figure + figcaption groups an image with its caption as one semantic
unit -->
<figure>
    <img src="images/photo.jpg" alt="...">
    <figcaption>A caption describing the photo above.</figcaption>
</figure>
```

alt is not optional. It's what screen readers announce, and it's what shows up if the file 404s. Two of your original images had `alt` written twice — an attribute can only appear once per element.

Audio — Live Demo

•

RENDERED OUTPUT — PRESS PLAY

Your browser does not support the audio element.

Multiple <source> tags let the browser pick whichever format it supports, top to bottom. autoplay only works combined with muted — browsers block audible autoplay to stop sites from blasting sound unexpectedly.

Video — Live Demo

RENDERED OUTPUT — PRESS PLAY

Your browser does not support the video tag.

iframe — Embedding Another Page

RENDERED OUTPUT

Not Every Site Allows Being Framed

Many sites (including most social media and banking sites) send a header telling browsers "refuse to display me inside an iframe," as an anti-clickjacking security measure. example.com above is a domain specifically reserved for documentation and demos, so it's a safe, reliable choice for practicing this tag.

CHAPTER 05

Lists & Tables

Your practice file covered ul/ol/dl and basic tables. Here's the full table anatomy (thead/tbody/tfoot/caption) you hadn't used yet, plus the correct nesting rule for lists.

Nested Lists — the Rule

**A Nested List MUST Live Inside an **

Your original file placed a nested `<ul>` directly inside the outer `<ul>`, as a sibling of the `<li>` items. That's invalid — a `<ul>`'s only legal direct children are `<li>` elements. The nested list has to sit inside one of those `<li>` items.

HTML — CORRECT

```
<ul>
    <li>Item 1</li>
    <li>Item 2
        <ul>
            <li>Subitem A</li>
            <li>Subitem B</li>
        </ul>
    </li>
</ul>
```

Full Table Anatomy

● RENDERED OUTPUT

Weekly Class Schedule

Day	Subject	Time
Monday	WS101	8:00-11:00
Wednesday	CC102	1:00-4:00

All times subject to change per COR

`thead`/`tbody`/`tfoot` group rows semantically (header, body, summary row) — your original table only used bare `tr`. `caption` titles the whole table. `colgroup`/`col` let you style entire columns at once instead of every cell individually.

CHAPTER 06

Grouping & Semantic Layout

The div-vs-span rule you already practiced, extended into the full set of layout tags — including `section` vs `article`, which trips up almost everyone at first.

BLOCK-LEVEL

** **

Stacks on its own line. Generic grouping box — no semantic meaning of its own. Can contain other block or inline elements.

INLINE

****

Flows in the middle of a sentence. Can NEVER legally replace a block element inside <h1>-<h6> or <p> — your original file had this bug twice.

section vs. article vs. div — The Decision

USE WHEN

<section>

Grouping a thematically related chunk of a page that would make sense with its own heading — e.g. an "About" section of a homepage.

USE WHEN

<article>

The content could stand completely alone and be copy-pasted elsewhere and still make sense — e.g. a blog post, a forum comment, a product card.

** is the fallback** — use it only when neither section nor article's meaning actually applies and you just need a styling/scripting hook.

Semantic Layout Tags — Live Demo

• RENDERED OUTPUT

<header> — intro/branding area

<nav> — a block of navigation links

<main> — the primary, one-per-page unique content

<aside> — tangential content, sidebars

<details> + <summary> — click to expand (zero JavaScript needed!)

This whole collapsible box is pure HTML — no script required. Great for FAQs.

<footer> — closing info, copyright

CHAPTER 07

Forms, End to End

Your practice file already covered nearly every input type that exists. This chapter adds `fieldset/legend` (grouping related fields, which you hadn't used) and a clean reference table of the validation attributes scattered across your form.

fieldset & legend — Grouping Related Fields

• RENDERED OUTPUT

Contact Details

Name:

Email:

`fieldset` draws a grouping box around related inputs; `legend` is its title. Purely organizational and accessibility-friendly — screen readers announce the legend before reading the grouped fields.

Validation Attributes — Full Reference

Attribute	Applies to	What it does
required	Any input	Blocks form submission until filled
minlength / maxlength	text, password, textarea	Character count limits
min / max	number, range, date, time	Value bounds
pattern	text, tel, etc.	A regex the value must match
placeholder	text-like inputs	Greyed-out example text — NOT a real value, disappears on typing
accept	type="file"	Hints which file types the file picker allows

Duplicate ids Your original form reused `id="color"` and `id="file"` twice each. Every id on a page must be unique — the <label for=""> connection breaks otherwise.

Wrong enctype A form with any `type="file"` input needs `enctype="multipart/form-data"`. Plain text encoding can't carry binary file data. Your form listed enctype twice — only one value per attribute is valid.

Label without for A <label> only auto-connects to its input when `for="id"` matches that input's `id` exactly — a typo silently breaks the click-to-focus behavior.

Global Attributes & Comments

These attributes work on almost *any* HTML tag, regardless of what element it is — they're not tied to one specific tag the way `href` is tied to <a>.

Attribute	Does what
id	Unique page-wide identifier — used by CSS, JS, and #anchor links
class	A label shared by multiple elements — how CSS targets groups of things
style	Inline CSS on this one element only (fine for practice; real projects use a separate CSS file)
title	Tooltip text shown on hover
lang	Declares the language of this element's content
hidden	Removes the element from the rendered page entirely
data-*	Custom attribute for storing your own data, read later by JavaScript (e.g. `data-userid="42"`)
tabindex	Controls keyboard Tab-key focus order
contenteditable	Makes an element directly editable by the visitor, no form needed
draggable	Makes an element draggable via native browser drag-and-drop

Two You Can Play With Right Now — Zero JavaScript

● RENDERED OUTPUT — TRY CLICKING INTO THE BOX, OR DRAGGING THE CHIP

contenteditable="true":

Click here and start typing — this text is directly editable.

draggable="true":

Drag me

CHAPTER 09

Accessibility, SEO Basics & What's Next

A quick professional-habits chapter, then the bridge into your next two lessons.

Accessibility — the 20% That Covers 80% of Cases

1. **Always write real `alt` text**
   Describe what's IN the image, not "image123.jpg." Decorative-only images can use `alt=""` (empty, not missing) so screen readers skip them cleanly.
2. **Use semantic tags over generic divs**
   A screen reader announces "navigation" for `<nav>` but says nothing useful for a `<div class="nav">` — same visual result, very different accessibility.
3. **Always pair <label> with its input**
   Either wrap the input inside the label, or use matching `for` / `id`. Screen readers and click-to-focus both depend on this connection.

Script Tag Recap

You already used a basic <script> block to react to a button click. Two attributes you'll see constantly once you write real JS files:

HTML

```
<!-- defer: downloads the script in the background, but waits to RUN it
until the HTML
```

```
     has finished parsing. Most common choice for scripts that touch the
page's elements. -->
<script src="app.js" defer></script>

<!-- async: downloads AND runs as soon as it's ready, may interrupt HTML
parsing.
     Best for independent scripts (ads, analytics) that don't touch the
page's own elements. -->
<script src="analytics.js" async></script>

<!-- noscript: shown ONLY if the visitor has JavaScript disabled -->
<noscript>This page needs JavaScript enabled to work fully.</noscript>
```

You're Ready for CSS and JavaScript

Structurally, this covers the full range of what HTML does: document setup, text, links, media, lists/tables, layout, forms, and the attributes that glue it all together. Your next lesson (CSS, WS101 Week 4) attaches a `<link rel="stylesheet">` to everything you just learned and controls how it looks. The one after that (JavaScript, Weeks 5-6) reaches into these same elements by `id` — like your `getElementById("reminder")` example — and controls how they behave. Nothing here gets thrown away; every chapter above is a building block both of those lessons will keep referencing.